

CENTRO COP · PLATAFORMA CLÍNICA ESCALABLE

# Informe técnico integral del aplicativo web y sus servicios

Evaluación estática de arquitectura, capacidades, integraciones, seguridad, operación y decisión de migración a Spring Boot.

**Repositorio revisado:** PA COP Escalable

**Fecha:** 31 de agosto de 2026

**Alcance:** frontends, API central, gateway, datos, IA, J48, despliegue y CI.

**Alcance de la evidencia.** Se revisó el código, los manifiestos, la orquestación y la documentación disponibles en el repositorio. No se afirma que un servicio esté operando en producción: el daemon Docker no estuvo disponible, Maven no está instalado en la consola y la ejecución de npm fue bloqueada por permisos del entorno.

# 1. Resumen ejecutivo

**Dictamen.** Centro COP es una plataforma clínica modular, con una API NestJS como núcleo transaccional, MongoDB y Redis, sitios Next.js para la experiencia actual y frontends Angular conservados como legado. La plataforma ya incorpora un microservicio Spring Boot para J48; mantener una arquitectura polígota es técnicamente justificado para los componentes de IA. No se recomienda migrar todo el backend a Spring Boot ahora: no hay un bloqueo técnico que lo exija y el cambio introducirá alto riesgo clínico y operativo. La prioridad es consolidar las rutas activas, retirar o aislar los stubs y cerrar brechas de producción.

**2****frontends actuales**

Next.js: web pública y dashboard clínico.

**2****frontends legado**

Angular: panel y web pública; aún compilados en CI.

**216****decoradores HTTP Nest**

Indicador estático de superficie de API; no equivale a rutas únicas.

**43****rutas FastAPI detectadas**

Servicios de IA y procesamiento complementario.

## Conclusiones prioritarias

1. La API NestJS cubre IAM, pacientes, historia clínica, citas, odontología, psicología, pagos, tenancy, analítica, IA y compatibilidad.
2. El gateway enruta API, sitio público, J48 y servicios IA; los servicios IA del perfil [ai](#) no forman parte del arranque [core](#).
3. Existen funciones explícitamente stub o de compatibilidad (diagnóstico/emoción y Fotos→3D en desarrollo); no deben anunciarse como capacidad clínica productiva hasta su integración y validación.
4. La configuración de producción valida secretos, CORS y webhooks, pero el compose mantiene credenciales de desarrollo que deben quedar fuera de cualquier despliegue real.

## 2. Arquitectura y flujo de información



| Componente           | Responsabilidad observada                                                                                 | Estado técnico                                                             |
|----------------------|-----------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------|
| web-public           | Landing, registro/inicio de sesión, cuenta, catálogo, reservas y pagos.                                   | <b>ACTUAL</b> Next.js 15 / React 19.                                       |
| web-dashboard        | Panel con roles, agenda, pacientes, psicología, J48, IA y administración.                                 | <b>ACTUAL</b> Next.js 15 / React Query / Socket.IO.                        |
| Frontend y PublicWeb | Equivalentes Angular para panel y web pública; cuentan con E2E Playwright.                                | <b>LEGADO ACTIVO</b> Duplican superficie funcional.                        |
| nest-migration       | API de negocio, RBAC, tenancy, persistencia Mongo, colas/Redis, pagos y orquestación IA.                  | <b>NÚCLEO</b> NestJS 10.                                                   |
| services/gateway     | Proxy inverso, health, límite multipart de 500 MB y timeouts para trabajos 3D.                            | <b>NÚCLEO</b> Nginx.                                                       |
| services/j48-python  | Inferencia J48 con scikit-learn, árbol y métricas; es el destino por defecto de <a href="#">J48_URL</a> . | <b>ACTIVO</b> perfil core.                                                 |
| services/j48-service | J48 Weka con Spring Boot, DTO, OpenAPI, token administrativo y explicación opcional.                      | <b>OPCIONAL</b> perfil <a href="#">j48-java</a> .                          |
| Servicios IA/3D      | Diagnóstico, emoción, recomendación, DICOM→GLB, profundidad→3D y ortodoncia.                              | <b>MIXTO</b> algunos requieren perfil <a href="#">ai</a> o proveedor real. |

### Datos y límites de confianza

MongoDB conserva entidades clínicas y administrativas; Redis soporta revocación de JWT y procesamiento. El [TenancyInterceptor](#) y los módulos de IAM indican segregación por organización. Debe comprobarse con pruebas de autorización por recurso, porque una estructura de módulos no demuestra por sí sola el aislamiento efectivo en todas las consultas.

### 3. Servicios y capacidades funcionales

| Dominio            | Servicios/evidencia                                                                             | Evaluación                                                                                                                                    |
|--------------------|-------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------|
| Identidad y acceso | IAM, JWT, refresh tokens, roles, guards, usuarios administrativos, bootstrap.                   | <b>IMPLEMENTADO</b> Requiere rotación de secretos y revisión de bootstrap antes de producción.                                                |
| Atención clínica   | Pacientes, citas, historia clínica, odontograma, tratamientos, radiografías y archivos 3D.      | <b>IMPLEMENTADO</b> Validar flujos E2E y permisos por rol/sede.                                                                               |
| Psicología         | Sesiones, evaluaciones, escalas, evolución, snapshots y scoring J48.                            | <b>IMPLEMENTADO</b> Los resultados son apoyo profesional, no diagnóstico autónomo.                                                            |
| Web pública        | Sedes, catálogo, agenda, reservas, reseñas, cuenta, Wompi; pagos Stripe/PayPal internacionales. | <b>IMPLEMENTADO</b> Webhook Wompi debe verificarse en sandbox y producción.                                                                   |
| IA clínica         | Proxy de diagnóstico/emoción/relapse, Medical AI, alertas, timeline, assistant y WebSocket.     | <b>PARCIAL</b> <code>api-compat</code> contiene respuestas stub para algunas rutas.                                                           |
| Imágenes 3D        | DICOM→GLB y proveedor imagen→3D asíncrono.                                                      | <b>PARCIAL</b> El proveedor Docker por defecto puede devolver un pato GLB de demostración; el código lo bloquea salvo habilitación explícita. |

#### J48: coexistencia Python y Spring Boot

La implementación Python es la ruta operativa actual. La Java implementa una alternativa Weka más estructurada: `/health`, `/info`, `/predict`, `/predict/explanation` y `/train`. Antes de reemplazar Python por Java hay que igualar explícitamente los contratos de árbol, métricas, entrenamiento, autenticación entre servicios y rendimiento. La coexistencia actual es válida, pero debe decidirse un único servicio propietario antes de escalar.

## 4. Seguridad, privacidad y operación

### Controles presentes

- Helmet, CORS, validación con whitelist y rechazo de campos no permitidos.
- Guard global de throttling, JWT y RBAC.
- Swagger solo fuera de producción.
- Health profundo para Mongo/Redis y liveness separado.
- Validación de producción para JWT, CORS, Mongo y webhook Wompi.

### Riesgos observados

- Valores de desarrollo en `compose.yaml`, incluidos password Mongo y `JWT_SECRET=change_me`.
- Mongo y Redis se publican en puertos locales: no exponerlos en un host compartido.
- CORS permite previews Vercel por defecto; conviene deshabilitarlo si no es necesario.
- Autenticación servicio-a-servicio no queda demostrada para todos los microservicios IA.

### Privacidad y uso clínico

El servicio J48 Java declara minimización de datos para la explicación generativa: solo envía variables desidentificadas y puede usar fallback local si el proveedor está deshabilitado. Para toda la plataforma se recomienda formalizar un inventario de datos, finalidad, retención, responsables, consentimiento, trazabilidad de accesos y procedimiento de incidentes conforme a la normativa aplicable de datos de salud en Colombia. Esta revisión no constituye certificación legal ni auditoría de cumplimiento.

### Observabilidad y continuidad

Hay health checks y logs de arranque, pero no se observó integración de APM/Sentry ni una estrategia comprobada de logs estructurados, backups/restore de Mongo, RPO/RTO o alertas de producción. Son requisitos de salida antes de operar datos clínicos reales.

## 5. Calidad y verificación

| Área              | Evidencia                                                                                                               | Resultado de esta revisión                                                               |
|-------------------|-------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------|
| CI                | <a href="#">.github/workflows/ci.yml</a> compila Nest, Angular, ambos Next y ejecuta smoke de entrenamiento J48 Python. | <b>CONFIGURADO</b> No se vio job de tests J48 Java ni E2E Docker.                        |
| J48 Spring Boot   | Pruebas MockMvc y de seguridad bajo <a href="#">services/j48-service/src/test</a> .                                     | <b>NO EJECUTADO</b> Maven no está instalado en la consola actual.                        |
| Construcción Node | Scripts <a href="#">build</a> para Nest, Angular y Next; lockfiles presentes.                                           | <b>NO EJECUTADO</b> npm falla al resolver una ruta fuera del área permitida del entorno. |
| Contenedores      | <a href="#">compose.yaml</a> define perfiles <a href="#">core</a> , <a href="#">ai</a> , <a href="#">j48-java</a> .     | <b>NO EJECUTADO</b> daemon Docker no disponible.                                         |

**Interpretación correcta.** La compilación y los tests definidos son una buena base, pero una definición de CI no es evidencia de que el último commit esté verde. Deben ejecutarse en un runner con dependencias y secretos de prueba controlados.

### Casos mínimos que faltan por automatizar

- Reserva pública completa y webhook Wompi firmado en sandbox.
- Aislamiento de datos de una organización frente a otra en todos los endpoints de pacientes, citas e historia.
- Autorización por rol y por sede para acciones sensibles.
- Fallos de Redis/Mongo, recuperación y revocación de token.
- Contrato de Nest con J48 Python/Java y regresión del modelo sobre dataset versionado.
- Flujo de imagen/DICOM contra un proveedor real, nunca contra el stub.

## 6. Decisión: ¿migrar todo a Spring Boot?

**Decisión recomendada: no realizar una migración big-bang.** La API NestJS ya concentra el dominio y los frontends consumen sus contratos. Migrarla completa a Java duplicaría 216 puntos HTTP potenciales, entidades Mongo, reglas RBAC, pagos, WebSocket, colas e integraciones. En una plataforma clínica, el riesgo de regresión supera el beneficio si no existe un requisito explícito de organización, soporte o rendimiento que NestJS no pueda cumplir.

| Opción                                        | Beneficio                                                      | Riesgo / coste                                                     | Recomendación |
|-----------------------------------------------|----------------------------------------------------------------|--------------------------------------------------------------------|---------------|
| Conservar NestJS + Spring Boot J48            | Entrega continua y tecnologías adecuadas para cada componente. | Dos stacks para operar y observar.                                 | AHORA         |
| Migrar solo J48 Python a Spring Boot          | Unifica la inferencia J48 en Weka/Java.                        | Debe alcanzarse paridad de contratos y validación de predicciones. | EVALUAR       |
| Extraer un nuevo módulo de Nest a Spring Boot | Prueba una transición controlada si hay motivo real.           | Gateway, observabilidad y contratos versionados obligatorios.      | PILOTO        |
| Migrar toda la API                            | Homogeneidad Java.                                             | Alto riesgo, largo plazo, escaso valor inmediato.                  | NO AHORA      |

### Ruta segura si se decide migrar J48

1. Definir contrato OpenAPI único y tests de consumidor para `/predict`, árbol, métricas y entrenamiento.
2. Ejecutar ambos servicios en paralelo, con entradas desidentificadas y comparación de resultados.
3. Versionar dataset, modelo, métricas, umbrales, fecha de entrenamiento y aprobación clínica.
4. Canary interno con rollback mediante `J48_URL`; no cambiar el comportamiento de pacientes sin aprobación.
5. Retirar Python solo cuando la paridad, seguridad y monitoreo estén demostrados.

## 7. Plan de remediación priorizado

| Prioridad | Acción verificable                                                                                    | Propietario sugerido   | Criterio de cierre                                                    |
|-----------|-------------------------------------------------------------------------------------------------------|------------------------|-----------------------------------------------------------------------|
| ALTA      | Eliminar secretos y credenciales de desarrollo de cualquier entorno productivo; rotar los existentes. | DevOps / seguridad     | Secret manager, escaneo de secretos y despliegue con valores únicos.  |
| ALTA      | Resolver u ocultar flujos IA que devuelven stubs y bloquear demostraciones 3D en producción.          | Producto / backend     | UX no promete capacidades no conectadas; proveedor real probado.      |
| ALTA      | Probar E2E reserva-pago y autorización multi-organización.                                            | QA / backend           | Suite CI verde con datos sintéticos y reporte de cobertura funcional. |
| MEDIA     | Elegir el único propietario de J48 y documentar contrato/versionado de modelo.                        | Arquitectura / clínica | ADR aprobada y rollback ensayado.                                     |
| MEDIA     | Implementar logs JSON, alertas, tracing, backups y prueba de restauración.                            | DevOps                 | Runbook con RPO/RTO y simulacro satisfactorio.                        |
| MEDIA     | Planificar retiro de Angular o declarar su soporte formal.                                            | Producto / frontend    | Mapa de rutas, fecha y reducción de duplicidad.                       |
| BAJA      | Añadir job CI de Maven para J48 Java y E2E contra compose aislado.                                    | DevOps                 | CI ejecuta pruebas sin secretos reales.                               |

## 8. Evidencia revisada

[README.md](#), [compose.yaml](#), [services/gateway/nginx.conf](#), [nest-migration/src/main.ts](#), [nest-migration/src/app.module.ts](#), [nest-migration/src/config/env.validation.ts](#), los módulos Nest, frontends, servicios IA, [services/j48-service](#), [.github/workflows/ci.yml](#) y documentación de arquitectura/despliegue. El informe refleja el estado del árbol de trabajo revisado el 31 de agosto de 2026.